

Table of Contents

Vision and Scope	1
1. Business requirements	2
1.1 Background	2
1.2 Business opportunity.....	2
1.3 Business objectives and success criteria.....	2
1.4 User needs	3
1.5 Business risks	3
2. Vision of the solution.....	3
2.1 Vision statement.....	3
2.2 Major features	3
2.3 Assumptions and dependencies.....	4
3. Scope and limitations	5
3.1 Scope of the initial release.....	5
3.2 Scope of subsequent releases	5
3.3 Limitations and exclusions	5
3.4 Constraints	6
4. Business context	6
4.1 Stakeholder profiles	6
4.2 Project priorities	6
4.3 Operating environment	7
5. Definition of terms.....	7
Revision history	7

Vision and Scope

Field	Value
Document ID	NOVA-VS-001
Version	0.3
Status	Draft
Owner	Laxmi Poudel

Field	Value
Date	2026-09-08

1. Business requirements

1.1 Background

Producing test cases from a requirement is high-volume, low-variance work. General-purpose LLM assistants generate competent test cases, but they retain no durable knowledge of a team's conventions between sessions.

The practical consequence is a recurring re-instruction cost. An engineer supplies the same constraints on every request: use Given/When/Then, include an unauthenticated case for every endpoint, tag priority P0 through P3, exclude UI cases from backend-only stories. Each correction is applied once and discarded. The generation is inexpensive; restating context indefinitely is not.

Existing mitigations rely on static configuration such as an extended system prompt or a style guide. These are unversioned, unmeasured, and degrade silently as they grow, and they provide no basis for answering whether output quality is improving.

1.2 Business opportunity

A system that captures corrections as durable, retrievable, and inspectable state can eliminate the re-instruction cost while remaining auditable. The differentiator is not generation quality but the closed feedback loop around it: extraction with provenance, scored retrieval, and a measured improvement metric gated in continuous integration.

1.3 Business objectives and success criteria

NOVA is a portfolio project. Objectives are expressed as demonstrated engineering capability.

ID	Objective	Success criterion
BO-1	Demonstrate that improvement is measured rather than asserted	Correction Recurrence Rate tracked across two or more runs on a fixed dataset, reported including regressions
BO-2	Demonstrate enforceable quality control	A deliberately degraded prompt version is blocked by an automated regression gate
BO-3	Demonstrate an inspectable learning mechanism	A stored memory can be opened, traced to its origin, deleted, and the resulting behaviour change observed
BO-4	Demonstrate defensible technical decision-making	Every architecturally significant decision recorded with alternatives and consequences
BO-5	Deliver a system useful to its author	Case acceptance rate measured over authentic personal usage

1.4 User needs

ID	Need	Current state
UN-1	Retain team conventions across sessions	Not met by existing assistants
UN-2	Receive output already in the team's format	Requires manual reformatting
UN-3	Determine whether the assistant is improving	No mechanism exists
UN-4	Understand why a given output was produced	Opaque
UN-5	Obtain complete negative, boundary, and authorization coverage	Inconsistent
UN-6	Avoid transmitting proprietary requirements to third-party APIs	Not addressed by hosted assistants
UN-7	Correct or remove a rule that produces poor output	No mechanism exists

1.5 Business risks

ID	Risk	Impact
BR-1	Correction extraction quality proves inadequate on a local model	The primary differentiator degrades to manually authored rules
BR-2	The headline improvement metric proves methodologically indefensible	Objective BO-1 cannot be evidenced
BR-3	Scope exceeds the available effort budget	Delivery incomplete at the point of presentation

Full treatment in [Risk Register](#).

2. Vision of the solution

2.1 Vision statement

For **software and test engineers** who **must produce test coverage from written requirements on a recurring basis**, NOVA is a **locally hosted developer agent** that **converts requirements into structured test plans and retains user corrections as durable, inspectable memory**. Unlike **general-purpose LLM assistants**, NOVA **applies prior corrections automatically, exposes the reasoning behind each output, and measures whether corrections stop recurring**.

2.2 Major features

ID	Feature	Priority
----	---------	----------

ID	Feature	Priority
FE-1	Requirement to structured test plan generation, schema-constrained	Must
FE-2	Correction capture at individual test case granularity	Must
FE-3	Extraction of corrections into durable memory with provenance and confidence	Must
FE-4	Scored memory retrieval with the scores exposed to the user	Must
FE-5	Memory management: browse, edit, pin, delete	Must
FE-6	Complete execution trace capture and viewer	Must
FE-7	Outcome-scored strategy selection across a fixed playbook set	Must
FE-8	Evaluation harness with a versioned dataset and CI regression gate	Must
FE-9	Pluggable inference providers with a local default	Must
FE-10	Single-command local deployment	Must

2.3 Assumptions and dependencies

ID	Assumption	Status
ASM-1	Local inference sustains schema fidelity at acceptable latency	Confirmed by measurement. See spike
ASM-2	A local model can extract reusable rules from freeform corrections at acceptable precision	Unverified. Highest project uncertainty
ASM-3	Container-based deployment satisfies the operational demonstration requirement	Accepted
ASM-4	The repository is published publicly, including evaluation data	Accepted
ASM-5	Development proceeds without external code review	Accepted constraint

ID	Dependency	Status
DEP-1	Ollama runtime	Installed, v0.33.3
DEP-2	Local model weights	<code>gemma3:4b</code> and <code>embeddinggemma</code> selected by measurement. 3.6 GB on disk
DEP-3	PostgreSQL with the <code>pgvector</code> extension	Not provisioned
DEP-4	Container runtime	Installed, daemon not running
DEP-5	Local embedding model	Not selected. Blocking, see ADR-0005

3. Scope and limitations

3.1 Scope of the initial release

- A single workflow: requirement to structured test plan and test cases
- Persistent memory derived from user corrections, subject to explicit confirmation before storage
- Retrieval scored on similarity, recency, and confidence, with scores surfaced in the interface
- Outcome-scored selection across four fixed playbooks with bounded exploration
- Complete execution trace persistence and inspection
- Evaluation harness: versioned dataset, deterministic checks, adversarial suite, CI regression gate
- Local-first inference behind a provider abstraction
- Container-based local deployment
- Single user, with a multi-tenant-capable schema and isolation enforced and verified from the outset

3.2 Scope of subsequent releases

Ordered by anticipated value relative to effort.

Release	Capability
2	Executable test code generation
2	Multi-user authentication and authorization
3	A second task type (defect report structuring) to demonstrate architectural generality
3	Issue tracker ingestion
4	Team-shared memory with a review workflow
4	Hosted deployment
5	Model fine-tuning, evaluated against the established baseline

3.3 Limitations and exclusions

ID	Excluded	Rationale
EX-1	Model fine-tuning or training of any kind	Deferred. The initial release produces the labelled dataset and baseline that fine-tuning would require
EX-2	Authentication and multi-user access	Schema is prepared; implementation deferred
EX-3	Hosted deployment or a public endpoint	Recorded demonstration substitutes
EX-4	Source repository or code context ingestion	Improves output at disproportionate cost to evaluability

ID	Excluded	Rationale
EX-5	Executable test code generation	Approximately doubles scope
EX-6	Issue tracker integrations	Not required to demonstrate the feedback loop
EX-7	Test execution or integration with a user's CI	Out of scope
EX-8	Real-time collaboration, mobile-optimized interface	Out of scope
EX-9	Autonomous learning without human oversight	Every memory write and configuration change is gated

3.4 Constraints

ID	Constraint	Source
CON-1	Approximately 130 to 180 engineering hours	Resource availability
CON-2	Single engineer, no external code review	Project structure
CON-3	Local inference by default, 6 GB VRAM ceiling	Measured hardware limit
CON-4	Local deployment only	Project decision
CON-5	Synthetic and open-source data only	Project decision
CON-6	No model fine-tuning in the initial release	Scope boundary
CON-7	Single user, with a multi-tenant-capable schema	Project decision

4. Business context

4.1 Stakeholder profiles

Stakeholder	Interest	Influence
Software Engineer (owner)	Delivery, technical demonstration, defensibility of design decisions	Full authority over scope, schedule, and acceptance
Primary user persona (test engineer on a feature team)	Reduced re-instruction cost, complete coverage, output in house format	Requirements source. Represented by the owner
External technical reviewer	Evidence of engineering judgment, measurement discipline, and testing rigour	Consumer of the artifact. No authority over scope

4.2 Project priorities

Dimension	Driver	Constraint	Degree of freedom
Features			Scope may be reduced against a pre-committed

Dimension	Driver	Constraint	Degree of freedom
			order
Quality	Evaluation harness and isolation guarantees are non-negotiable		
Schedule		Approximately three months at 10 to 15 hours per week	
Cost		Zero marginal inference cost required	

4.3 Operating environment

Single workstation. Windows 11 host. Containerized application services. Inference served by a host-resident runtime with GPU acceleration, 6 GB VRAM. No network egress in the default configuration.

5. Definition of terms

See [SRS section 1.4](#).

Revision history

Version	Date	Author	Change
0.1	2026-09-08	Laxmi Poudel	Initial draft
0.2	2026-09-08	Laxmi Poudel	CON-7 recorded in the constraints table
0.3	2026-09-08	Laxmi Poudel	DEP-2 resolved